

Teaching a Small Model to Manage an AI Agent's Memory

Thesis proposal (updated October 2026)

The problem

AI agents built on large language models can only read a limited amount of text at once. In a long task, like a support chat that runs for days, the history quickly grows past that limit. Something has to decide what the model keeps in front of it, what gets filed away for later, and what gets thrown out. Most systems use simple fixed rules, like "keep the most recent messages." These rules are cheap, but they often throw away the one fact that ends up mattering.

My question

Can a very small, separate model learn to make these memory decisions better than fixed rules, without retraining the language model itself?

My approach

I built a test framework where a frozen language model answers questions while a separate "memory controller" decides what to keep, archive, delete or retrieve. The controller is tiny (about 27,000 parameters), so it trains on a laptop. Because my tasks are simulated, I know afterwards which facts were actually needed. I use that hindsight to build a teacher for the controller to copy, and then try reinforcement learning (GRPO) on top.

What I have found so far

1. **The choice of teacher matters most.** Copying a perfect, all-knowing teacher works badly (34% of questions right at a tight budget), because the student can't see what the teacher sees. Copying a teacher that only marks which choices cost nothing gets 85%, beating every fixed rule I tested.
2. **It works with a real language model.** With Qwen2.5-7B reading, the controller does as well as showing the model everything, while using only 40% of the text.
3. **The main remaining problem was search.** Most mistakes were two-step questions where the second fact was never found. A simple "follow the clue" search raised accuracy from 85% to 92%.
4. **Reinforcement learning adds only a little,** about one more point.

What I will do next

1. Test on two public benchmarks of long conversations, LoCoMo and LongMemEval. Early results show that controllers trained only on my simulated task don't transfer to real conversations, so I am now training them on benchmark data and testing on held-out questions.
2. Study memory that costs something to store, where filing everything away is no longer free.
3. Work out when a small learned controller is worth it, and when a simple rule is enough.

Why it matters

If a small, cheap model can manage memory well, any agent could use it without retraining a large model. If it can't, knowing why is still useful for anyone building agents that need to remember things.